

↖



↖

↖

↖

↖

↖

↖

↖

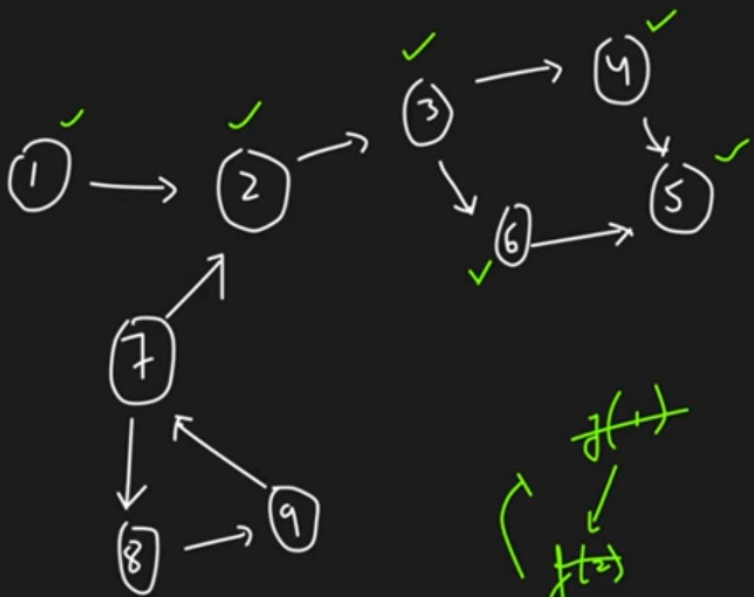
↖

↖

↖

↖

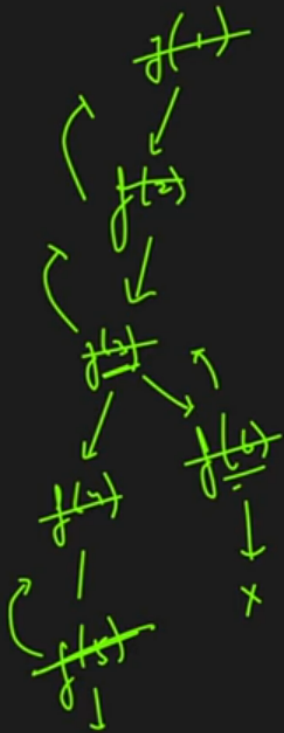
Detect a cycle in Directed Graph (DFS)



```

for (i = 1 → 9)
{
    if (!vis(i))
    {
        if (isPrime(i))
        {
            vis(i) = 1;
            return T
        }
    }
}

```



adjacency List

$$1 \rightarrow 2$$
 $2 \rightarrow 3$

3 → 4, 6

$$4 \rightarrow \underline{5}$$
$$5 \rightarrow //$$
$$6 \rightarrow 5$$
$$7 \rightarrow 2, 8$$
$$8 \rightarrow 9$$
$$9 \rightarrow 7$$

vis

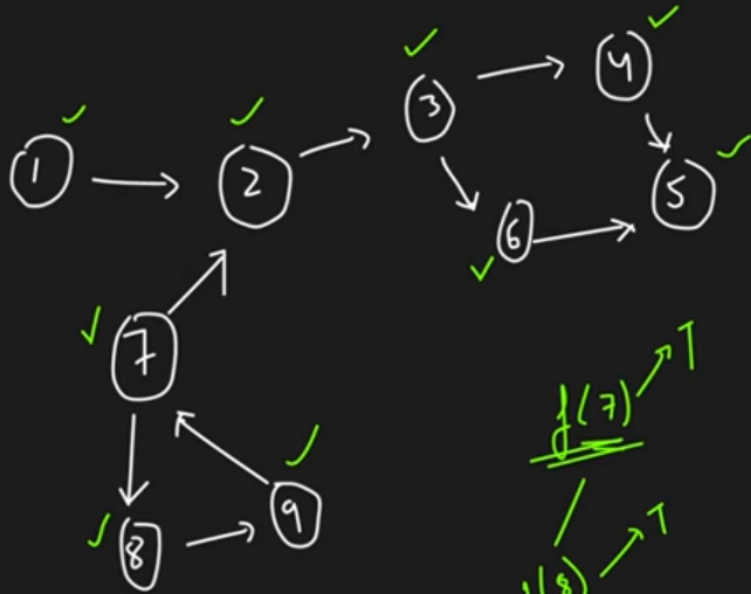
1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9

dis vis

1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9

Detect a cycle in Directed Graph (DFS)

$TC \rightarrow O(N+E)$
 $SL \rightarrow O(2N)$
 $ASL \rightarrow O(N)$



```

for(i=1 to 9)
{
  if(!vis(i))
  {
    if(helper(i))
    {
      return T;
    }
  }
}

```

$f(7) \rightarrow T$
 $f(8) \rightarrow T$
 $f(9) \rightarrow T$

$vis[node] = 1$
 $dfsvis[node] = 1$

adjacency List

$1 \rightarrow 2$
 $2 \rightarrow 3$
 $3 \rightarrow 4, 6$
 $4 \rightarrow 5$
 $5 \rightarrow$
 $6 \rightarrow 5$
 $7 \rightarrow 2, 8$
 $8 \rightarrow 9$
 $9 \rightarrow 7$

vis									
1	2	3	4	5	6	7	8	9	
1	2	3	4	5	6	7	8	9	
1	2	3	4	5	6	7	8	9	
1	2	3	4	5	6	7	8	9	
1	2	3	4	5	6	7	8	9	

```

6- class Solution {
7- private:
8-     bool checkCycle(int node, vector<int> adj[], int vis[], int dfsVis[]) {
9-         vis[node] = 1;
10-        dfsVis[node] = 1;
11-        for(auto it : adj[node]) {
12-            if(!vis[it]) {
13-                if(checkCycle(it, adj, vis, dfsVis)) return true;
14-            } else if(dfsVis[it]) {
15-                return true;
16-            }
17-        }
18-        dfsVis[node] = 0;
19-        return false;
20-    }
21- public:
22-     bool isCyclic(int N, vector<int> adj[]) {
23-         int vis[N], dfsVis[N];
24-         memset(vis, 0, sizeof vis);
25-         memset(dfsVis, 0, sizeof dfsVis);
26-
27-         for(int i = 0; i < N; i++) {
28-             if(!vis[i]) {
29-                 // cout << i << endl;
30-                 if(checkCycle(i, adj, vis, dfsVis)) {
31-                     return true;
32-                 }
33-             }
34-         }
35-         return false;
36-     }
37- }
38- // } Driver Code Ends

```

